FuzzyTutor

Technical Project Report, Fuzzy-System Definition, Evaluation, and User Manual

19 August 2026

Executive summary

We built FuzzyTutor as a web-based adaptive tutor for programming concepts and as a joint project for the Fuzzy Logic and Systems course and the User Experience and Artificial Intelligence course. The central idea was to keep academic performance and learning friction separate, so the system uses two complementary inference engines:

1. a first-order Takagi–Sugeno ANFIS implementation that predicts a continuous Knowledge Mastery score; and
2. a Mamdani fuzzy inference system that estimates System Cognitive Friction from observable learning behavior.

The backend combines both outputs when it selects the next uncompleted task. In the interface, the learner sees mastery and friction gauges, a focus-state label, a short support message, and the reason for the difficulty change. A short satisfaction survey appears after every five completed tasks and remains due after a refresh.

We used Django and Django REST Framework for the backend, React and Vite for the frontend, SQLite for assignment-scale persistence, and Docker Compose so the same setup can be demonstrated on another machine. The frontend only consumes JSON contracts; fuzzy rules and answer keys stay on the backend.

We did not have a historical learner dataset. For that reason, the ANFIS training pipeline starts from a deterministic synthetic dataset with five plausible behavioral profiles. This lets us demonstrate training and inference, but it is not evidence about a real student population. We use a fixed 80/20 holdout split so the evaluation rows are not used to fit the model parameters.

Requirements and scope

The implemented system addresses the final joint project definition as follows:

Requirement	Implementation
Personalized fuzzy decisions	Continuous mastery/friction estimates drive task difficulty and support messages
Fuzzy sets and membership functions	Explicit triangular and shoulder functions in both engines
Fuzzy rules	Five ANFIS activation rules and six core

Requirement	Implementation
	Mamdani rules plus explicit code-workspace and mixed-signal coverage guards
Defuzzification	ANFIS normalized weighted TSK output; Mamdani centroid of aggregated output area
ANFIS extension	Trainable first-order consequent parameters with a versioned model artifact
Seven programming modules	Lists, arrays, dictionaries, classes, inheritance, exceptions, and control flow
MCQ and code-writing tasks	Private answer-key MCQs and non-graded code workspace tasks
Dynamic adaptation	Harder, equivalent, or easier uncompleted task chosen from the curriculum
Explainable AI	Engine memberships, active rules, scores, focus state, recommendation, and friendly reason
Feedback loop	Persistent micro-survey every five completed tasks and telemetry export
Decoupled web application	Django REST API and React frontend in separate containers

The backend is intentionally assignment-scale. We chose anonymous session tokens and SQLite instead of adding production identity, authorization, and distributed infrastructure. This keeps the implementation small enough to inspect during the presentation while still exercising the fuzzy system end to end.

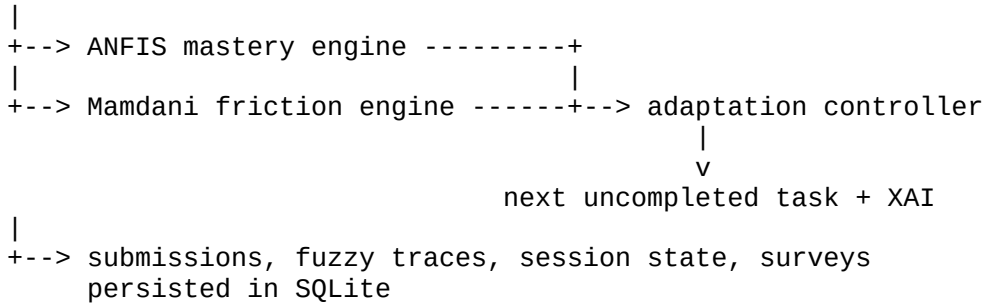
Design choices and trade-offs

Three decisions shaped the implementation. We do not automatically grade code-writing tasks: completion is behavioral evidence, while correctness stays unknown. The browser also never decides correctness or hint counts. The backend derives both from private task data and stored hint events, which avoids leaking answer keys and makes the telemetry easier to trust. Finally, the interface shows a short explanation before the full rule trace. Raw memberships helped us debug the system, but they belong in the expandable demonstration view rather than the learner's main workspace.

System architecture

The operational data flow is:

```
React learning workspace
        |
        | task answer + time + completion + assistance
        v
Django REST submission endpoint
        |
        +--> private task metadata and backend MCQ correctness
```



The backend is separated into three Django applications:

- `core`: health and shared service concerns;
- `learning`: curriculum, learner sessions, submissions, surveys, telemetry, and adaptation; and
- `fuzzy`: membership functions, ANFIS, Mamdani inference, training, evaluation, and synthesis.

Data model

`LearnerSession` stores the anonymous session token, current module/task, rolling mastery/friction aggregates, completed-task count, and latest recommendation.

`ModuleProgress` stores module-local mastery and friction, attempt count, active/terminal status, exit reason, and completion time. Module-local evidence prevents strong performance in one topic from silently skipping an untouched topic.

`TaskSubmission` stores task metadata at the moment of submission, elapsed and relative response time, server-derived assistance count, maximum revealed hint level, module-state snapshots, module-exit outcome, completion ratio, backend-derived MCQ correctness, and the answer payload. Code-task correctness remains null because code is not automatically graded.

`HintEvent` stores each immutable progressive hint reveal with its session, task, level, hint type, content snapshot, and elapsed reveal time. A uniqueness constraint permits each of the three levels to be revealed only once per session task. Session restoration returns revealed hints but never exposes unrevealed hint content.

`FuzzyEvaluationLog` stores a reproducible input snapshot, complete engine trace, both crisp outputs, focus state, recommendation, and support message.

`MicroSurveyResponse` stores satisfaction, perceived difficulty, confidence, optional comments, and the exact five-task milestone. A uniqueness constraint prevents duplicate responses for one session milestone.

Learning content and interaction design

The curated task bank contains 105 tasks: 15 in each module, with five tasks in each of three difficulty bands. It contains 53 MCQs and 52 code-writing tasks:

Band	Level	Task metric weight	Estimated cognitive load
Foundation	1	35	Low
Intermediate	2	55	Medium
Advanced	3	75	High

MCQ choices are returned to the browser, but correct answers remain private before an attempt. Code tasks return starter code but not an answer guide. Every task also has a private explanation for read-only review.

Every task defines three private, distinct support levels: a conceptual cue, a strategy, and a scaffold. The hint endpoint reveals them sequentially and persists each reveal. Normal catalog and session task payloads contain no unrevealed hint content.

Code submissions are deliberately not executed or graded. This prevents compiler/test feedback from becoming an additional anxiety signal. The backend derives completion instead of trusting a browser value: an explicit skip is 0, a valid MCQ choice is 1, and meaningfully edited code is 1. Blank code and code equivalent to the normalized starter template are rejected with guidance to edit or skip. A code response is never stored as correct merely because it is non-empty.

Shared fuzzy membership functions

For a value x, the triangular membership function with left point a, peak b, and right point c is:

$\mu_{triangle}(x; a,b,c) = 0$ outside $[a,c]$; $(x-a)/(b-a)$ for $a < x < b$; and $(c-x)/(c-b)$ for $b \le x < c$.

The left-shoulder function is 1 at and below a, decreases linearly between a and b, and is 0 at and above b. The right-shoulder function is its mirror: 0 at and below a, increases linearly, and is 1 at and above b.

All final crisp scores are clamped to the interval [0,100].

ANFIS cognitive-performance engine

Inputs and output

The cognitive engine predicts Knowledge Mastery on [0,100]. Its evidence is:

- Task Metric Weight on [0,100];
- Historical Grade Average, represented operationally by the session's rolling mastery, on [0,100];
- Completion Ratio on [0,1];
- a correctness/performance signal; and
- task type, used only for a small incomplete-code penalty feature.

For MCQs, correctness is computed against the private backend key and maps to performance evidence 100 (correct) or 20 (incorrect). For code tasks, correctness is unknown; the

conservative performance signal is `40 + 25C`, where C is completion ratio. Therefore even a completed code response contributes at most 65 and never receives MCQ-like correctness credit.

Premise membership sets

Variable	Linguistic set	Function and parameters
Challenge	Foundation	left shoulder (35, 60)
Challenge	Intermediate	triangle (35, 58, 82)
Challenge	Advanced	right shoulder (65, 85)
History	Low	left shoulder (35, 60)
History	Medium	triangle (45, 68, 86)
History	High	right shoulder (72, 90)
Completion	Low	left shoulder (0.20, 0.65)
Completion	Medium	triangle (0.35, 0.70, 0.98)
Completion	High	right shoulder (0.75, 1.00)
Performance	Weak	left shoulder (35, 65)
Performance	Emerging	triangle (45, 70, 90)
Performance	Strong	right shoulder (75, 95)

Rule layer

Rule activation uses fuzzy AND = minimum and fuzzy OR = maximum.

1. **Secure prior mastery:** high history AND high completion AND strong performance.
2. **Developing mastery:** (medium history AND high completion) OR (high history AND emerging performance).
3. **Productive challenge:** advanced challenge AND high completion AND strong performance.
4. **Fragile progress:** (medium history AND medium completion) OR (emerging performance AND medium completion).
5. **Knowledge gap:** (low history AND weak performance) OR (low completion AND weak performance).

Each rule has a first-order TSK consequent:

$$f_i = p_i H + q_i C + r_i P + s_i W + t_i K + b_i$$

where H is history, C is completion percentage, P is the performance signal, W is task weight, K is the incomplete-code penalty, and b is a bias. The crisp mastery prediction is the normalized weighted mean:

$$\text{Mastery} = \Sigma(w_i f_i) / \Sigma(w_i)$$

where w_i is the activation strength of rule i. This corresponds to the core ANFIS layers: premise fuzzification, rule firing, normalization, consequent calculation, and output aggregation. The premise definitions are fixed for transparency; the consequent parameters are trained.

The five pedagogical rules form a compact rule base rather than the full Cartesian product of all

premise sets. If a valid combination activates none of them, a visible mixed-evidence coverage guard routes the combination through the trained `developing_mastery` consequent. Its strength is the minimum of the best-supported linguistic set for challenge, history, completion, and performance. The API trace reports `coverageGuardUsed=true`, so the engine never silently substitutes a non-rule prediction.

Synthetic training data

The deterministic generator creates 180 records across five profiles: strong, struggling, improving, overchallenged, and fast-incorrect. Each row includes task weight, history, response-time ratio, assistance, completion, task type, correctness state, confidence, perceived difficulty, and a target mastery label. MCQ rows contain Boolean correctness; every code row has null correctness and uses the same conservative completion evidence as runtime inference.

The synthetic target is a declared weighted teaching heuristic, not hidden ground truth:

Target = 0.42(performance evidence) + 0.24(completion) + 0.14(speed) + 0.12(confidence) + 0.08(history) + difficulty adjustment.

The seed is 42. A deterministic split holds out 20% of records before gradient fitting. The versioned model was trained for 650 epochs at learning rate 0.00002.

Holdout evaluation

Metric	Default consequent baseline	Trained model
Holdout records	36	36
MAE	8.368	3.523
RMSE	10.813	4.367
R^2	0.610	0.936

The RMSE improvement is 6.445 points on held-out synthetic samples. Two consecutive training runs produced the byte-identical artifact SHA-256 `1daf10f8cb248484cad85c788a18c2a4f21f0ce270a3a1c138e76e8f81b9c68e`. These results demonstrate that the implemented training pipeline learns its declared synthetic relationship. They do not establish effectiveness for real learners; real-data calibration is future work. Training and evaluation use an explicit `synthetic_only` source mode, so unrelated learner sessions cannot change the versioned artifact or its reported holdout metrics.

Mamdani cognitive-friction engine

Inputs and premise sets

The behavioral engine predicts System Cognitive Friction on [0,100]. It uses response time relative to the task baseline, assistance interactions, completion ratio, and whether the workspace is a code task.

Variable	Set	Function and parameters
Relative response time	Low	left shoulder (0.45, 0.95)
Relative response time	Normal	triangle (0.65, 1.00, 1.45)
Relative response time	High	right shoulder (1.15, 2.10)
Assistance	Low	left shoulder (0.0, 1.5)
Assistance	Medium	triangle (0.5, 2.0, 3.0)
Assistance	High	right shoulder (2.0, 3.0)
Completion	Incomplete	left shoulder (0.15, 0.55)
Completion	Partial	triangle (0.25, 0.60, 0.95)
Completion	Complete	right shoulder (0.70, 1.00)

The output linguistic sets are:

Friction set	Function and parameters
Low	left shoulder (15, 35)
Moderate	triangle (20, 42, 64)
High	triangle (48, 72, 90)
Severe	right shoulder (72, 92)

Mamdani rules

1. Low time pressure AND low assistance AND complete → low friction.
2. Normal time AND complete → low friction.
3. (Normal time AND medium-or-high assistance) OR (high time AND complete) → moderate friction.
4. Partial completion AND (normal time OR medium-or-high assistance) → moderate friction.
5. High time AND (medium assistance OR high assistance) → high friction.
6. (Incomplete AND high time) OR (incomplete AND high assistance) → severe friction.
7. A code workspace activates moderate friction at strength 0.35 to represent its additional interaction load without treating it as failure.
8. If none of the six behavioral rules activates for a valid MCQ signal combination, a visible mixed-signal coverage guard activates moderate friction. Its strength is the minimum of the best-supported time, assistance, and completion memberships.

Inference and centroid defuzzification

Antecedents use minimum for AND and maximum for OR. Each active rule clips its consequent membership function at the rule strength. Consequents are aggregated pointwise with maximum over a [0,100] universe sampled at 0.25-point resolution.

The crisp result is the centroid of the aggregated output area:

Friction = Σ(x μ_aggregated(x)) / Σ(μ_aggregated(x)).

The API trace returns the input memberships, active rule strengths, coverage-guard state, output-set definitions, aggregated area, and the explicit method name `centroid_of_aggregated_output_area`. Operational inputs always produce a non-zero

aggregated fuzzy area; the numerical fallback remains only as a defensive safeguard for invalid internal calls.

Synthesis and adaptation controller

The controller maps both crisp scores to a calm user-facing state:

- **Focused & Steady:** friction < 25 and mastery ≥ 65;
- **Needs Support:** friction < 55 when the steady condition is not met; and
- **Frustrated:** friction ≥ 55.

The next difficulty decision is:

Condition	Recommendation	Action
Mastery ≥ 75 and friction < 35	Increase or hold high tier	Target one level harder, capped at advanced
Mastery < 45 or friction ≥ 55	Reduce difficulty and support	Target one level easier, floored at foundation
Mixed evidence	Hold current tier	Stay near the current level

The selector consumes this single recommendation rather than independently reinterpreting the crisp scores. It never repeats a completed task. During the first six attempts in a module it balances MCQ and code tasks while remaining near the fuzzy target difficulty. After each attempt from the sixth onward, the module is mastered when module mastery is at least 75, friction is below 40, and at least two of the latest three MCQs are correct. Otherwise selection continues through the 15-task bank. Bank exhaustion advances the learner without falsely labelling the module mastered. Untouched modules are never skipped.

Focus and action are separate explainable dimensions: mild friction can produce `Needs Support` while high mastery still requests advancement. Support text is generated from both dimensions, so this combination recommends advancing with a hint or explanation available. The response reports `requestedDirection`, the actually applied `direction`, and a nullable `constraintApplied`. At foundation and advanced boundaries, the actual action is `hold` and the response names the difficulty floor or ceiling instead of claiming an impossible change.

Global session and module-local aggregates use a smoothing update: 65% prior aggregate plus 35% latest engine output. Curriculum completion means that all seven module-progress records are terminal, including mastery exits and exhausted banks.

Explainability and feedback

Every submission response contains:

- Knowledge Mastery and System Cognitive Friction;
- focus state, recommendation, and support message;
- requested and applied next-task direction, any floor/ceiling constraint, and a plain-language reason;
- input snapshot; and

- optional detailed memberships and active-rule trace.

The primary interface should show the friendly interpretation rather than raw rule identifiers. A detailed trace can be placed in an expandable assessment/debug section.

At every five completed tasks, the backend exposes `surveyDue=true`. The oldest unanswered milestone remains due across refreshes. A survey records 1–5 satisfaction, perceived difficulty, and confidence scores plus an optional comment. Duplicate responses for the same milestone are rejected. The export endpoint joins these labels to task and model telemetry for later analysis.

Because no live study has yet been conducted, this report makes no fabricated claims about student or educator satisfaction. The implemented feedback mechanism, persistence, and export contract support a future evaluation with both groups. A suitable study would compare perceived difficulty with model friction, examine satisfaction before and after adaptations, and collect educator judgments about recommendation appropriateness.

REST API contract

Method and path	Purpose
`GET /api/health/`	Service status
`GET /api/learning/modules/`	Seven modules and task counts
`GET /api/learning/tasks/`	Public, answer-safe task catalog
`POST /api/learning/sessions/`	Create anonymous learning session
`GET /api/learning/sessions/{token}/`	Restore session and pending survey state
`GET /api/learning/sessions/{token}/review/`	Review skipped and incorrect attempts without retry
`POST /api/learning/hints/`	Reveal and persist the next progressive hint
`POST /api/learning/submissions/`	Persist response, run both engines, adapt next task
`POST /api/learning/micro-surveys/`	Save the currently due survey milestone
`GET /api/learning/export/training-data/`	Export joined telemetry for assignment analysis
`POST /api/fuzzy/evaluate/`	Standalone engine demonstration without persistence
`GET /api/docs/`	Typed Swagger/OpenAPI documentation

Malformed or unknown module identifiers, invalid navigation directions, stale/non-current task submissions, invalid MCQ choices, blank or unchanged code, client-supplied completion, correctness, assistance, or answer metadata, duplicate surveys, and unknown tokens receive 4xx responses rather than server errors. Submission history and ANFIS training labels are built by the backend; public clients cannot inject `skipped`, `synthetic`, or `targetMastery` metadata.

User manual

Starting the system

1. Install Docker with Docker Compose support.
2. From the project directory, run `docker compose up --build`.
3. Open `http://localhost:5173` for the tutor.
4. Open `http://localhost:8000/api/docs/` to inspect or demonstrate the backend contracts.

The frontend and backend run in separate containers. Database migrations run when the backend container starts.

Learner workflow

1. Start or restore a learner session.
2. Select the answer to an MCQ or write a response in the code editor.
3. If support is needed, reveal a conceptual cue, then a strategy, then a scaffold. Each level is persistent and contributes to the assistance signal.
4. Submit the task. The code editor does not run automated tests.
5. Read the mastery/friction gauges and the short support message.
6. Continue with the backend-selected next task. The adaptation reason explains why its difficulty was chosen.
7. After each five-task milestone, answer the satisfaction, perceived-difficulty, and confidence survey.
8. Continue until the curriculum-complete state is reached.

If a page refresh occurs, the stored session token can be used to restore the current task, aggregates, completion count, and any unanswered survey milestone.

Instructor/developer workflow

- Inspect engine inputs and active rules in the submission response's `engineTrace`.
- Use the training-data export for tables and offline analysis.
- Reproduce the synthetic model by running the seed, train, and evaluate management commands documented in the repository README.
- Use Swagger to verify frontend payloads. The browser must never send `isCorrect`, `assistanceInteractions`, `answerPayload`, `taskMetricWeight`, or private solution data during the session workflow.

Verification and evaluation

Automated backend verification includes:

- private three-level task hints, sequential reveal, restoration, and server-derived assistance telemetry;

- ANFIS strong/weak evidence and output-bound tests;
- exhaustive operational-grid checks that every ANFIS inference activates at least one rule;
- deterministic synthetic-row generation;
- training-artifact creation and holdout metadata;
- Mamdani behavioral ordering and centroid method tests;
- exhaustive operational-grid checks that every Mamdani inference has a non-zero aggregated output area;
- answer-safe public task contracts;
- invalid-query 400 responses;
- rejection of client correctness, assistance, answer metadata, and stale tasks;
- server-derived completion, valid-choice enforcement, and meaningful code-edit validation;
- null correctness for non-graded code tasks;
- 105-task distribution, answer privacy, non-repeating selection, balanced evidence, mastery exit, bank exhaustion, review, and legacy-session progression; and
- persistent, non-duplicated survey milestones.

The verified suites contain 42 backend tests and 6 frontend tests. Django system checks, migration-drift checks, OpenAPI schema validation, and the production frontend build also pass.

Limitations and future improvements

The main limitations are:

1. Synthetic records demonstrate the model pipeline but do not replace real learner data. With consent and appropriate anonymization, later work should retrain and recalibrate membership parameters from actual sessions.
2. ANFIS premise membership parameters are fixed for interpretability; only consequent parameters are trained. A larger study could compare this transparent variant with hybrid premise/consequent learning.
3. Mastery is tracked globally and per module, but not yet per individual concept tag. A longer deployment could maintain finer-grained knowledge components.
4. Anonymous tokens and the telemetry export are suitable for a local assignment demonstration. A deployed multi-user system would require authentication, role-based export access, retention rules, and a production database.
5. The user study remains future work. The existing micro-survey and export mechanisms are ready to support student and educator evaluation without inventing results.

Conclusion

The prototype meets the joint assignment goals in a workflow that is practical to demonstrate: a learner can complete tasks, ask for help, see an explanation, and receive an adapted next task. The project defines and tests the fuzzy variables, rules, inference methods, trained ANFIS consequents, and evidence behind each decision. Until it is tested with real learners, however, the synthetic results show that the pipeline works as designed, not that it improves learning outcomes.